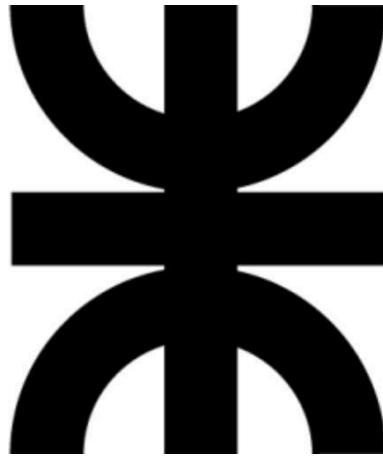




UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA
Ingeniería en Sistemas de Información

Cátedra Ingeniería y Calidad de Software
Comisión 4k3

Trabajo Práctico N°6

TDD- 6 - Test Driven Development (TDD)

Grupo 15

Integrantes:

- Galiano, Tomas 85824
- Reyna, Teodoro 89891
- Chudnosky, Paula Milagros 89385
- Vera, Agustín Alejandro 83085
- Yoles Trucco, Nicolás 81404
- Pavon, Florencia 89539
- Capellan, Jorge Gabriel 82595
- Poltawcew, Ivan Mijail 85470
- Nass, Franco David 88534

Docentes:

- Ing. Laura Covaro
- Ing. Cecilia Massano
- Ing. Georgina González
- Ing. Garnero Constanza
- Ezequiel Izaguirre

Introducción

En el presente trabajo se aborda la implementación de la historia de usuario **“Comprar Entradas”** como un caso de estudio práctico. El objetivo principal es aplicar la disciplina de **Desarrollo Dirigido por Pruebas (Test Driven Development - TDD)**, utilizando las pruebas unitarias como el artefacto principal que guía el diseño y la implementación de la lógica de negocio.

Con esta práctica buscamos reforzar los conceptos de TDD y la arquitectura limpia (Puertos y Adaptadores) vistos en clase, trasladándolos a un entorno real. La correcta aplicación de este enfoque resulta clave para mantener la calidad, asegurar la robustez y facilitar la mantenibilidad del producto de software durante todo su ciclo de vida.

Desarrollo

Metodología TDD Aplicada

El desarrollo se adhirió estrictamente al ciclo clásico de TDD (Red-Green-Refactor), asegurando que no se escribiera código de lógica de negocio sin que existiera primero una prueba unitaria que fallara y definiera el comportamiento esperado.

Etapa	Descripción	Resultado
 Red (Rojo)	Se escriben los tests unitarios (con xUnit y Moq) antes de implementar la lógica funcional.	Tests fallan (NotSupportedException o Assert fallido).
 Green (Verde)	Se implementa el código mínimo y suficiente en la capa de dominio (TicketingService) para que pasen los tests.	Tests pasan exitosamente.
 Refactor (Refactorizar)	Se mejora la estructura, nombres y se eliminan duplicaciones en el código de dominio, sin romper los tests.	Código limpio y probado.

Criterios de Aceptación y Pruebas Implementadas

Se definieron los Criterios de Aceptación (CA) de la historia de usuario, los cuales se tradujeron directamente en pruebas unitarias.

Criterio de Aceptación	Prueba Unitaria Implementada (xUnit)
Solo usuarios registrados pueden comprar.	Rechaza_Compra_Si_Usuario_No_Esta_Registrado
La fecha de visita debe ser actual o futura.	Rechaza_Compra_Si_Fecha_Es_Pasada
El parque debe estar abierto ese día (Cierra Lunes/Martes).	Rechaza_Compra_Si_Parque_Esta_Cerrado
Se pueden comprar hasta 10 entradas por transacción.	Rechaza_Compra_Si_Cantidad_Entradas_Es_Mayor_A_10
Debe especificarse la forma de pago .	Rechaza_Compra_Si_No_Se_Elige_Forma_De_Pago
Si es tarjeta , redirige a URL simulada.	Acepta_Compra_Con_Tarjeta_Y_Returna_Url_Redireccion
Si es efectivo , marca "pago en boletería".	Acepta_Compra_En_Efectivo_Y_Marca_Pago_En_Boleteria
En ambos casos, envía confirmación por email (mockeada).	Envia_Email_Correcto_En_Compra_Efectivo

Arquitectura y Tecnologías

La arquitectura se basa en Puertos y Adaptadores para aislar el dominio. Las tecnologías seleccionadas permitieron mantener esta separación

Componente	Tecnología / Artefacto	Propósito
Lenguaje	C# (.NET 8)	Plataforma de desarrollo principal.
Pruebas	xUnit	Framework de pruebas unitarias para validar los CA.
Mocks	Moq	Librería para simular dependencias externas (Puertos).
Backend	ASP.NET Core Minimal API	Adaptador de entrada (endpoint <code>POST /tickets/purchase</code>).
Dominio	<code>EcoHarmony.Tickets.Domain</code>	Contiene la lógica de negocio pura y los "Puertos" (interfaces).
Frontend	Express server HTML + JS (Fetch API)	Adaptador de demo para consumir la API.
Documentación	Swagger UI	Documentación interactiva de la API.
Builder	Docker	Herramienta para exportar el proyecto con todas sus dependencias y facilitar instalacion

Página Principal

Comprar Entradas

Datos de compra

Email del comprador

El email es obligatorio.

Fecha de visita



La fecha de visita es obligatoria.

Forma de pago



La forma de pago es obligatoria.

Agrega entradas a la izquierda
para registrar las edades de los
visitantes.

Entradas

Pase Regular

Precio: \$5.000

Cantidad

Debes seleccionar al menos 1 entrada.

Pase VIP

Precio: \$10.000

Cantidad

Subtotal Regular: \$ 0

Subtotal VIP: \$ 0

Total: \$ 0

Total 0 de 10 entradas.

Comprar

**Error: Revisa los campos
marcados.**

Link al repositorio

<https://github.com/paulachudnosky/TDD-Comprar-Entradas-G15>

Conclusión

En este trabajo práctico, desarrollamos la implementación completa de la funcionalidad "Comprar Entradas", aplicando de manera sistemática la metodología TDD. La creación de pruebas unitarias previo a la codificación nos permitió poner en práctica los conceptos de diseño guiado por pruebas, generando una lógica de negocio (TicketingService) robusta, aislada y completamente verificable.

Asimismo, la estructura del proyecto (basada en Puertos y Adaptadores) demostró ser fundamental para el éxito de TDD, facilitando el aislamiento de dependencias mediante *mocks*. Este ejercicio resultó fundamental para afianzar nuestras competencias en TDD y xUnit, brindándonos una base sólida para generar software de alta calidad, con menos errores y mayor facilidad de mantenimiento.